PRÁCTICA N.º 7 — HOJA DE EVIDENCIAS

Pruebas automatizadas de un microservicio

COM-600 · Microservicios · Gestión 2/2026 · Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ing. de sistemas
Fecha de entrega	8/10/2026
Sistema operativo, versión de Node y de Docker	Docker v29.7.2, node v24.18.1, Windows 10 64bits
Repositorio (URL)	https://github.com/David-Franz-Soliz-Ortega/MicroServiciosDFSO/tree/573dfbde26ee926e53b3d43a35f09d21e97ac3bf/practicas/practica%207-pruebas

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

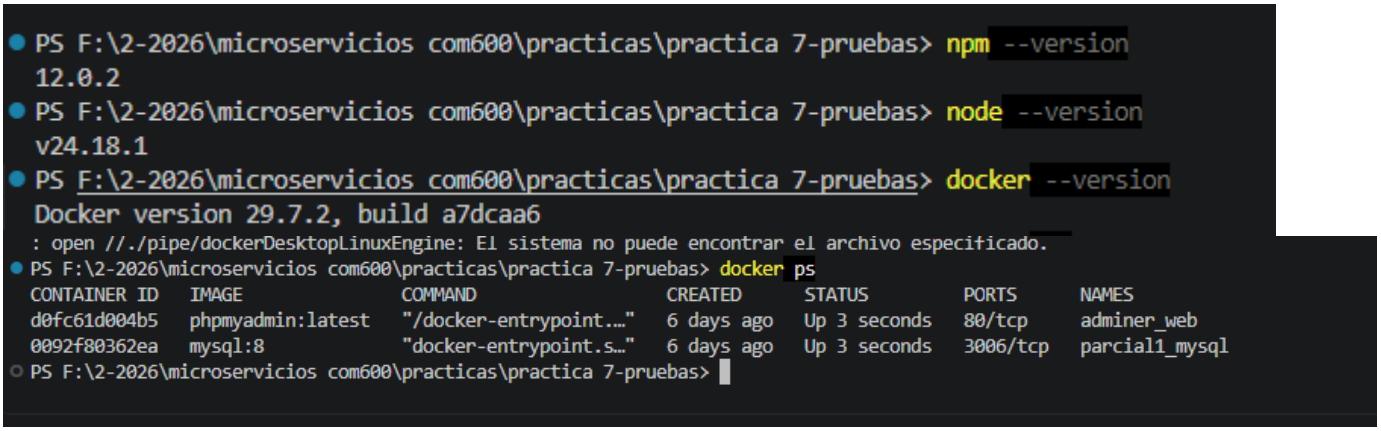
Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno	1, 2	1	X
Laboratorio 1 — La primera prueba unitaria	3	2	X
Laboratorio 2 — Casos borde y errores: donde viven los defectos	4, 5	3	X
Laboratorio 3 — Cobertura: qué mide y qué no	6, 7	4	X
Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor	8, 9	—	X
Laboratorio 5 — Integración real con un contenedor efímero	10, 11	5	X
Laboratorio 6 — Prueba de contrato entre dos servicios	12, 13	6	X
Laboratorio 7 — Dejar el proyecto listo para el pipeline	14, 15	7	X

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Verificación del entorno

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal.



Pregunta N.º 1. ¿Por qué esta práctica necesita Docker si lo que vamos a escribir son pruebas y no contenedores de la aplicación?

Para garantizar que el entorno de pruebas sea idéntico para todos. Permite levantar dependencias efímeras al momento de probar y destruirlas al terminar, sin instalar nada en tu sistema local.

Captura N.º 2: salida de npm install mostrando los paquetes instalados y el archivo package.json creado.

```
PS F:\2-2026\microservicios com600\practicas\practica 7-pruebas> npm install express mongodb

added 80 packages, and audited 81 packages in 18s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS F:\2-2026\microservicios com600\practicas\practica 7-pruebas> npm install --save-dev jest supertest
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have
 been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 296 packages, and audited 377 packages in 1m

80 packages are looking for funding
  run `npm fund` for details

19 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
npm warn install-scripts 2 packages had install scripts blocked because they are not covered by allowScripts:
npm warn install-scripts   @parcel/watcher@2.6.0 (install: node scripts/build-from-source.js)
```

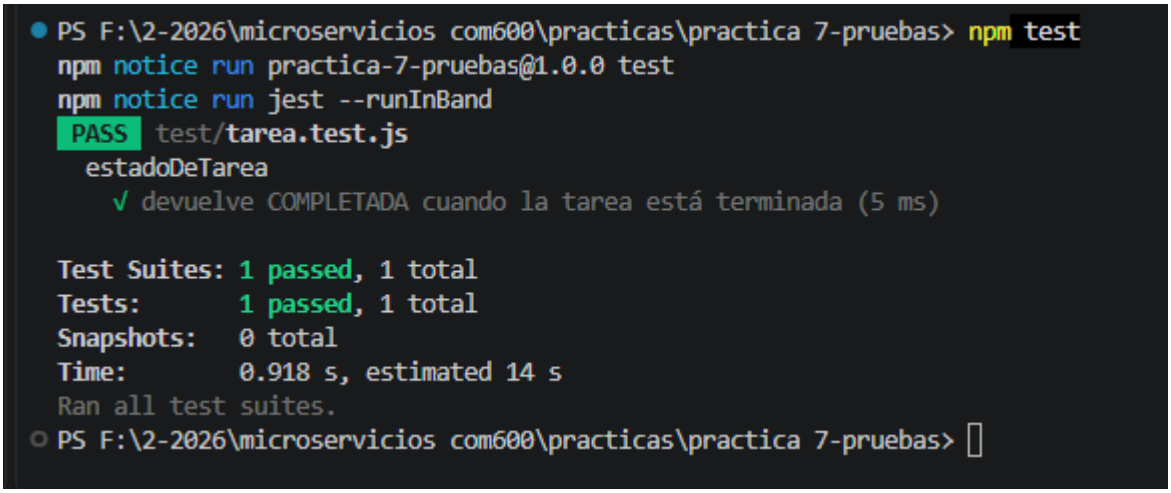
---

package.json > { } scripts > abc test

```
{
  "name": "practica-7-pruebas",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Depurar
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1",
    "mongodb": "^7.7.0"
  },
  "devDependencies": {
    "jest": "^30.5.2",
    "supertest": "^7.3.1"
  }
}
```

Laboratorio 1 — La primera prueba unitaria

Captura N.º 3: salida de npm test con la primera prueba pasando en verde.

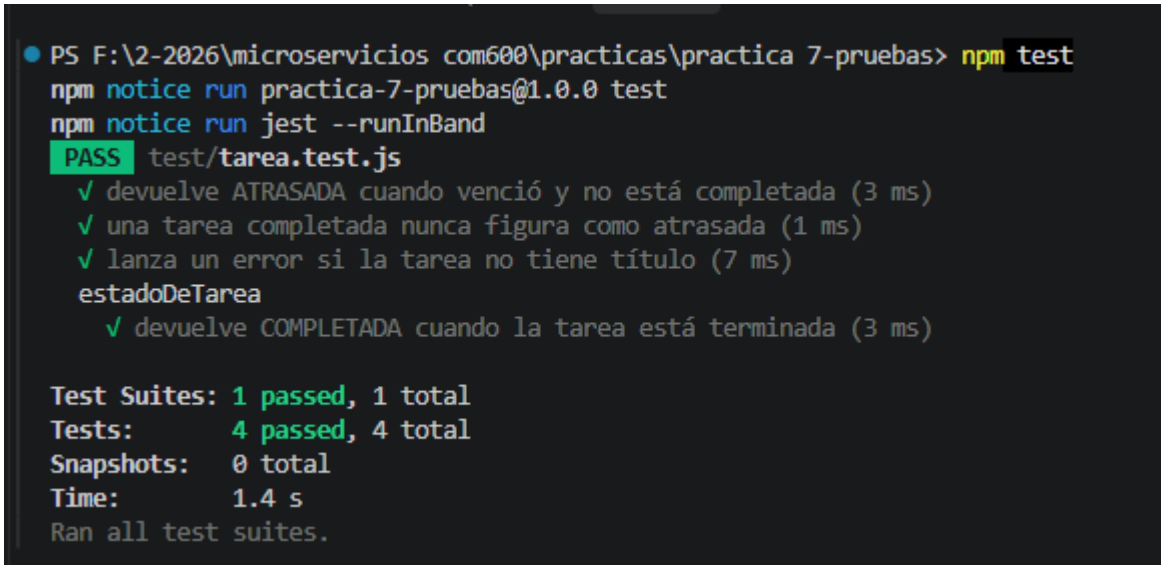


Pregunta N.º 2. El nombre de la prueba es «devuelve COMPLETADA cuando la tarea está terminada» y no «test1». ¿Qué gana el equipo cuando esa prueba falla dentro de seis meses?

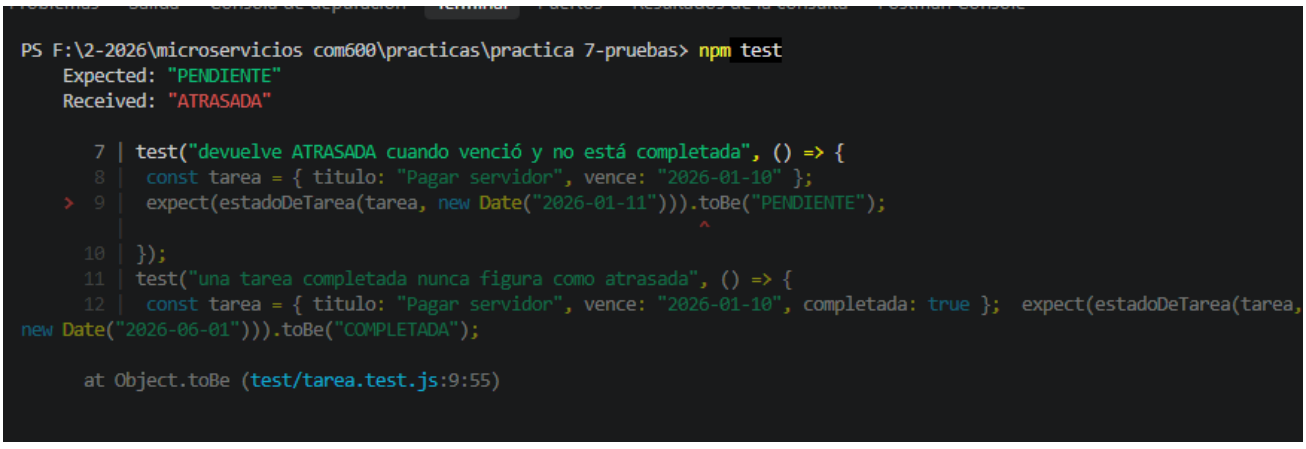
El equipo gana tiempo y contexto. Si la prueba falla en el futuro, el nombre indica exactamente qué regla del negocio se rompió, sin necesidad de que el programador tenga que leer y descifrar el código de la prueba.

Laboratorio 2 — Casos borde y errores: donde viven los defectos

Captura N.º 4: salida de npm test con las cuatro pruebas del bloque en verde.



Captura N.º 5: salida de la prueba en rojo, mostrando el valor esperado y el recibido.

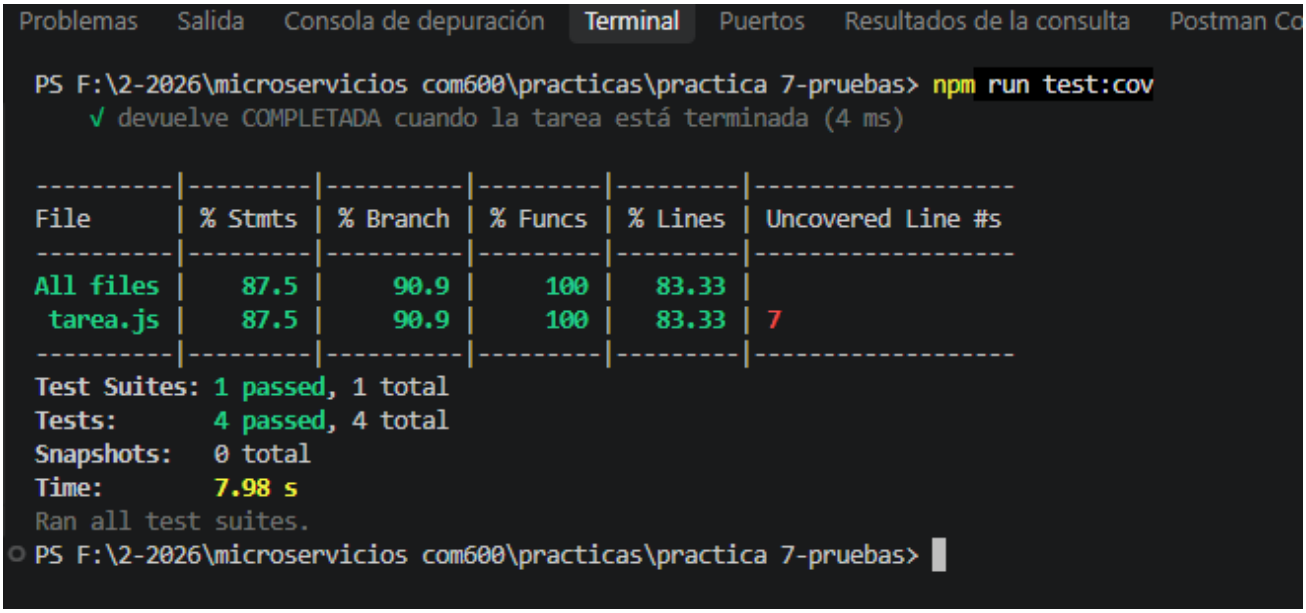


Pregunta N.º 3. Una prueba que nunca ha fallado no demuestra nada. ¿Cómo puede comprobar que una prueba realmente verifica lo que dice verificar?

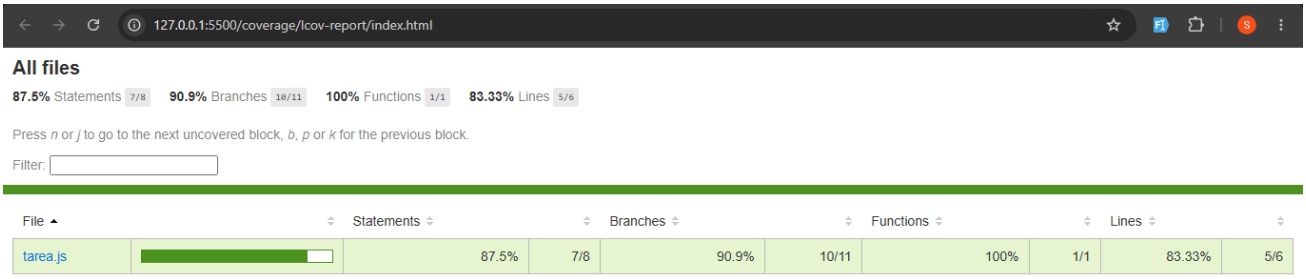
Alterando intencionalmente el código de producción (por ejemplo, cambiando un + por un − o comentando una línea) para asegurarse de que la prueba detecte el error y cambie a estado fallido ("rojo").

Laboratorio 3 — Cobertura: qué mide y qué no

Captura N.º 6: tabla de cobertura impresa en la terminal.



Captura N.º 7: reporte HTML de cobertura mostrando el archivo src/tarea.js con sus líneas coloreadas.

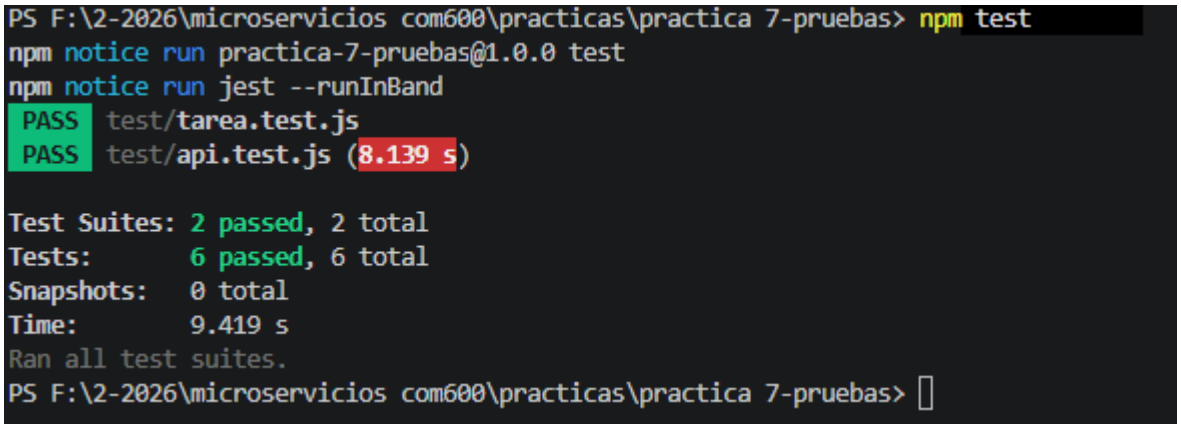


Pregunta N.º 4. Escriba una prueba que ejecute todas las líneas de estadoDeTarea pero que no verifique nada útil (por ejemplo, sin ningún expect que compare el resultado). ¿Qué cobertura reporta? ¿Qué conclusión saca sobre la cobertura como criterio de calidad?

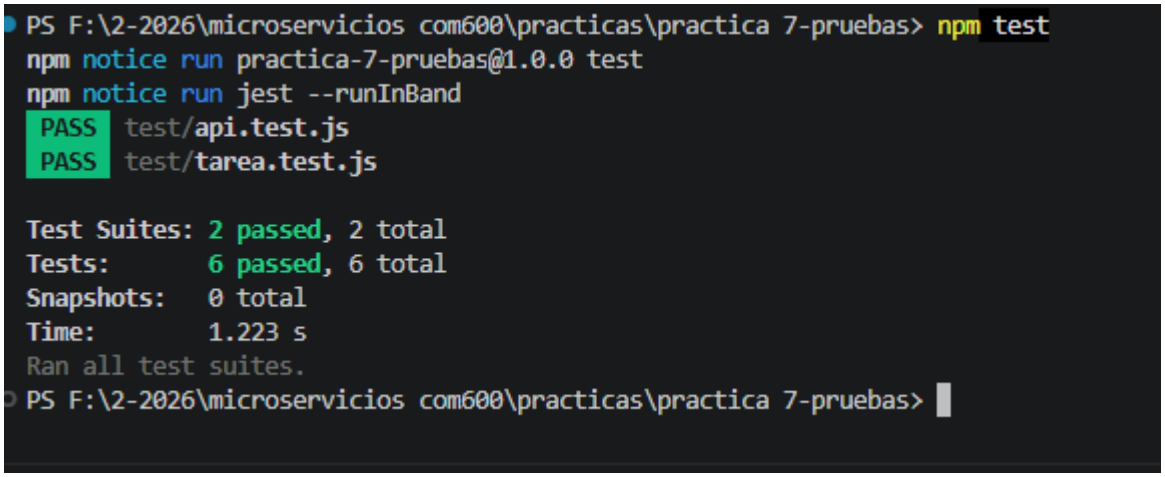
Reportará una **cobertura del 100**, pero la prueba es inútil. La conclusión es que la cobertura de código es solo una métrica de cantidad, no de calidad; un porcentaje alto no garantiza que el sistema esté libre de errores si no hay expects válidos.

Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor

Captura N.º 8: salida de npm test con las pruebas de la API en verde.

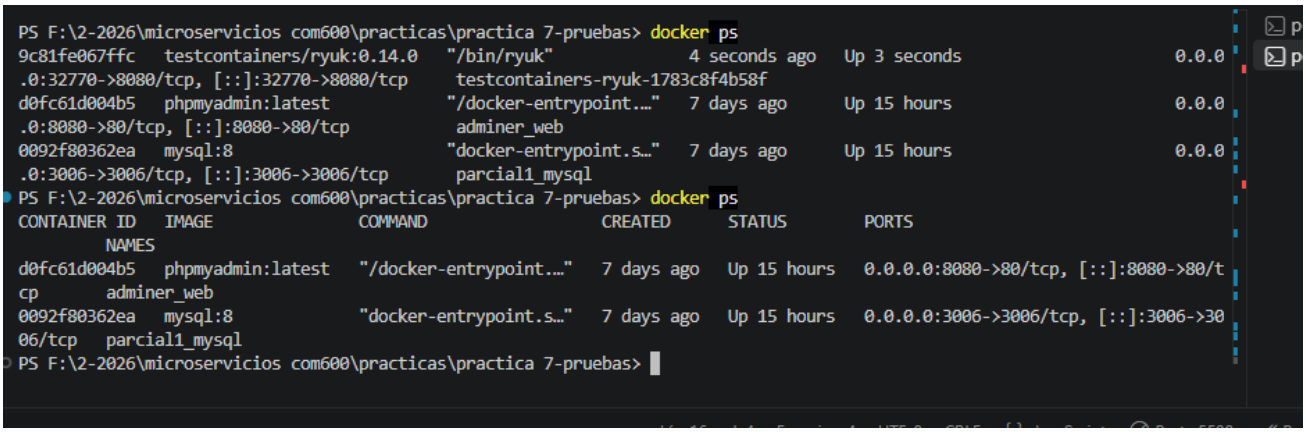


Captura N.º 9: salida completa de la suite con el tiempo total de ejecución visible.

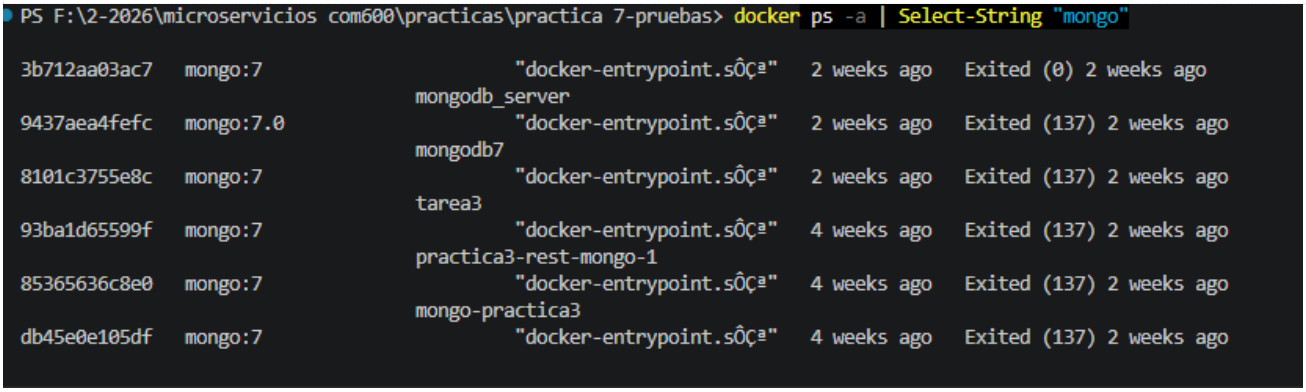


Laboratorio 5 — Integración real con un contenedor efímero

Captura N.º 10: las dos terminales: la suite ejecutándose y docker ps mostrando el contenedor efímero levantado por la prueba.



Captura N.º 11: salida mostrando que el contenedor de la prueba ya no existe después de terminar la suite.

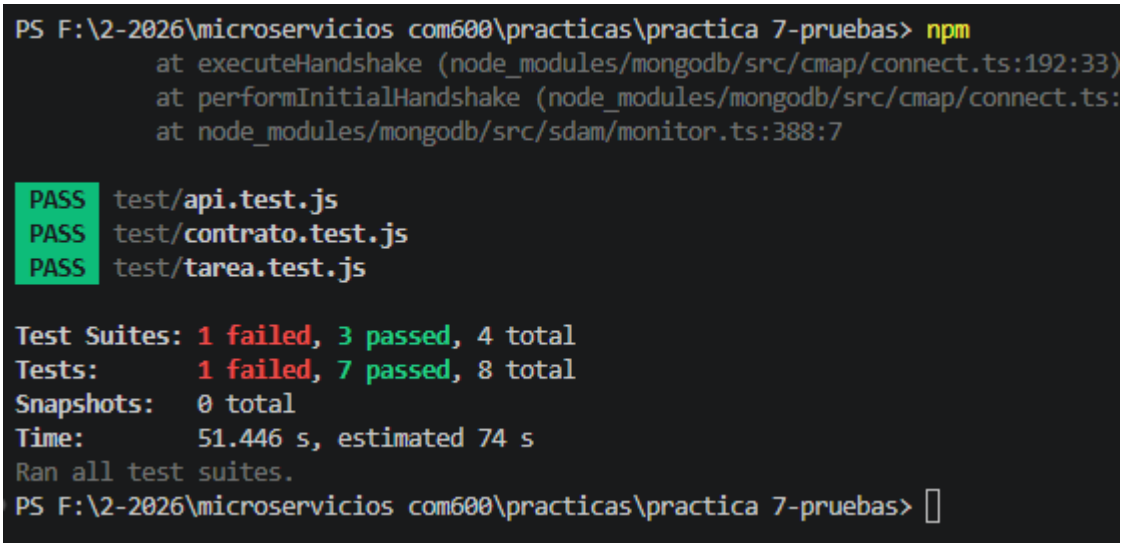


Pregunta N.º 5. ¿Qué problema concreto evita este enfoque frente a la alternativa de que cada estudiante instale MongoDB en su propia máquina y las pruebas apunten a esa instalación?

Previene errores por diferencias de versiones de MongoDB entre estudiantes, configuraciones de puertos ocupados y asegura que la base de datos esté completamente vacía antes de cada prueba.

Laboratorio 6 — Prueba de contrato entre dos servicios

Captura N.º 12: la prueba de contrato en verde junto al resto de la suite.



Captura N.º 13: la prueba de contrato fallando y señalando el campo faltante.

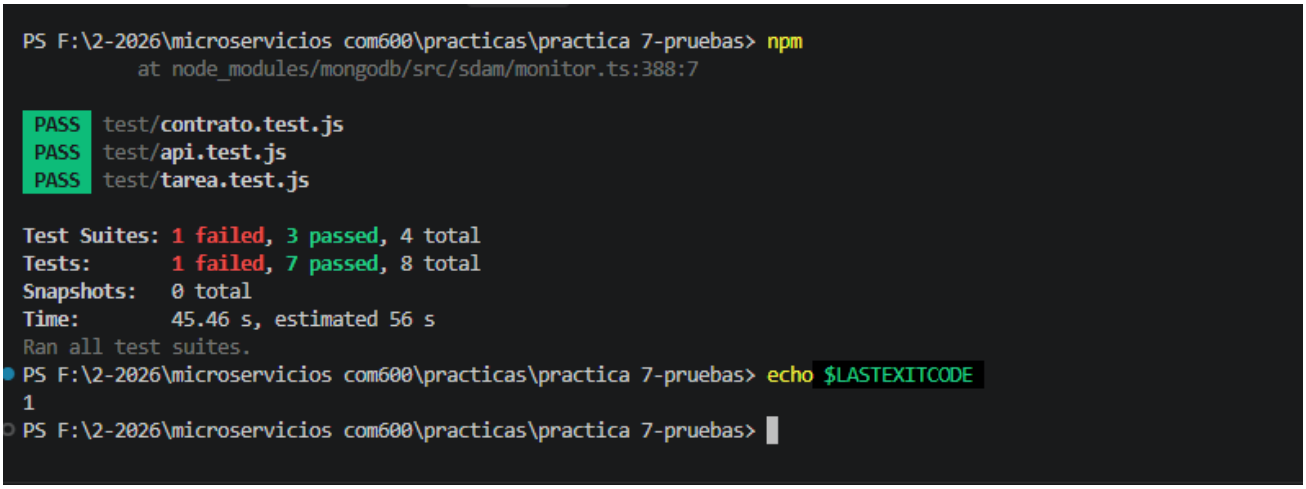


Pregunta N.º 6. Las pruebas unitarias del servicio de Tareas siguieron en verde con ese cambio. ¿Por qué? ¿Qué tipo de defecto detecta la prueba de contrato que ningún otro nivel detecta?

Siguieron en verde porque las pruebas unitarias evalúan la lógica interna simulando las respuestas externas. Las pruebas de contrato detectan errores de integración y comunicación: avisan cuando un servicio cambia la estructura de sus datos y rompe la comunicación con otro servicio que dependía de esa estructura.

Laboratorio 7 — Dejar el proyecto listo para el pipeline

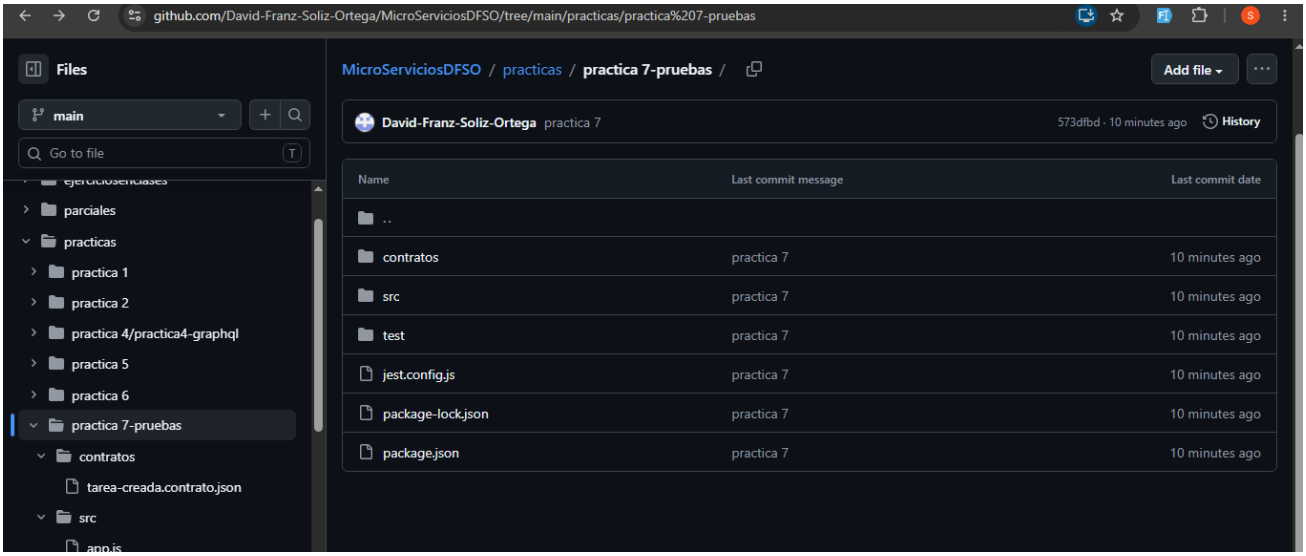
Captura N.º 14: los dos códigos de salida: 0 con la suite en verde y distinto de 0 con la suite en rojo.



Pregunta N.º 7. Si una prueba es inestable (a veces pasa y a veces falla), ¿qué le ocurre al pipeline y qué le ocurre a la confianza del equipo? ¿Qué haría usted con esa prueba?

Un pipeline inestable se bloquea al azar, retrasando los pases a producción. Esto destruye la confianza del equipo, quienes empezarán a ignorar las alertas pensando. Lo que se debe hacer es aislarla de inmediato hasta que se corrija el código, o eliminarla si no aporta valor real.

Captura N.º 15: la página del repositorio en GitHub mostrando la carpeta practica7-pruebas con tests/, contratos/ y el README.



Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

Hacer funcionar el contrato y un error relacionado al generar en Docker el contenedor temporal

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

El error con el contenedor de docker que crea el código a generar

¿Qué haría distinto si tuviera que repetir esta práctica?

Revisar que versión de la imagende mongodb tengo instalado para ejecutar las pruebas

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

_____	_____
Firma del estudiante	Fecha